



FACULDADE DE CIÊNCIAS DA UNIVERSIDADE DO PORTO

Mestrado de Segurança Informática · Segurança e Aplicações de Hardware Confiável · 2025/26

Data Lakes Seguros

Confidencialidade e Integridade com Trusted Execution Environments

Índice

Índice	2
Glossário	4
1. Introdução	5
2. Contexto	5
2.1 Data Lakes	6
2.2 Arquitetura Base	7
3. Fundamentos Técnicos	7
3.1 Intel Software Guard Extensions (SGX)	7
3.2 Atestação Remota e DCAP	8
3.3 Criptografia Aplicada ao Enclave	9
4. Estado da Arte	9
4.1 Sistemas de Analytics Confidencial com SGX	9
4.2 Computação Confidencial no Domínio Médico	10
4.3 Trusted Execution Environments: Comparativo	10
4.4 Ataques Conhecidos ao SGX e Mitigações	11
5. Arquitetura do Sistema	11
5.1 Visão Geral	11
5.2 Fluxo de Ingestão de Dados	12
5.3 Processamento de Queries	12
5.4 Modelo de Confiança e Garantias de Segurança	13
6. Implementação e Desenvolvimento	13
6.1 Stack Tecnológico	13
6.2 Enclave Mínimo - Fase 1	13
6.3 Suporte Multi-Contribuidor - Fase 2	14
6.4 Motor de Queries - Fase 3	14
6.5 Atestação Remota - Fase 4	14
7. Protótipo Implementado	15
7.1 Stack Tecnológica	15
7.2 Estrutura do Código	16
7.3 Estruturas de Dados e Interface	16
7.4 Fluxo de Atestação Remota	16
7.5 Cifra e Upload de Dados	17
7.6 Motor de Queries	17
7.7 Execução do Protótipo	17
7.8 Simplificações Face à Arquitetura Alvo	17
8. Avaliação de Segurança e Desafios	18
8.1 Análise de Ameaças por Componente	18
8.2 Limitações do EPC e Paginação	18
8.3 Side-Channels e Mitigações	19
8.4 Ataques por Inferência Estatística	19
9. Conclusão	20
10. Referências	21

Glossário

CDM (Common Data Model): Modelo padronizado para organizar dados de diferentes fontes.

EHDEN (European Health Data Evidence Network): Projeto europeu de padronização de dados clínicos.

IMI (Innovative Medicines Initiative): Parceria europeia que financia inovação na saúde.

OMOP (Observational Medical Outcomes Partnership): Padrão internacional para harmonizar bases de dados de saúde.

SMPC (Secure Multi-Party Computation): Protocolo que permite processar dados de várias fontes sem que estas os revelem entre si.

TEE (Trusted Execution Environment): Área isolada e segura dentro de um processador para execução protegida de dados.

AES-GCM (Advanced Encryption Standard - Galois/Counter Mode): Algoritmo que cifra e autentica dados simultaneamente.

DCAP (Data Center Attestation Primitives): Modelo da Intel para verificar a integridade de enclaves localmente em datacenters.

ECDH (Elliptic-Curve Diffie-Hellman): Protocolo para estabelecer chaves secretas entre duas partes de forma segura.

EPC (Enclave Page Cache): Memória RAM protegida e cifrada onde o enclave armazena código e dados.

HKDF (HMAC-based Extract-and-Expand Key Derivation Function): Função usada para derivar chaves de segurança a partir de um segredo partilhado.

HMAC (Hash-based Message Authentication Code): Código que garante que uma mensagem não foi alterada durante a transmissão.

MRENCLAVE: Identidade criptográfica única (hash) que representa o código exato de um enclave.

PCCS (Provisioning Certificate Caching Service): Serviço local que guarda certificados da Intel para acelerar a atestação DCAP.

PCK (Provisioning Certification Key): Chave única do processador usada para assinar provas de que o hardware é genuíno.

SGX (Software Guard Extensions): Tecnologia da Intel que cria "ilhas" de memória isoladas (enclaves) no processador.

TCB (Trusted Computing Base): Conjunto mínimo de componentes (hardware/software) em que o sistema confia para ser seguro.

TLS (Transport Layer Security): Protocolo que garante a segurança da comunicação entre o cliente e o servidor.

1. Introdução

A crescente adoção de data lakes em ambientes cloud trouxe um conjunto de desafios de segurança sem precedentes. Estes repositórios de grande escala, concebidos para armazenar e processar dados em múltiplos formatos e esquemas, tornaram-se infraestruturas críticas nos setores da saúde, finanças e investigação científica. A sua arquitetura aberta e colaborativa, que constitui a sua principal vantagem, é simultaneamente a sua maior vulnerabilidade: ao agregar dados de múltiplas fontes e ao expô-los a múltiplos utilizadores, o data lake multiplica a superfície de ataque e exige mecanismos de proteção robustos e multidimensionais.

No domínio médico, esta tensão é particularmente aguda. Um repositório partilhado entre hospitais, institutos de investigação e autoridades de saúde pública pode acelerar enormemente a descoberta científica, desde a deteção precoce de surtos epidémicos à avaliação da eficácia de medicamentos à escala populacional. Contudo, os mesmos dados que habilitam estas análises são, na sua essência, registos pessoais sensíveis: diagnósticos, resultados laboratoriais, histórico de tratamentos. A sua exposição, mesmo parcial, pode ter consequências graves para os titulares dos dados e configura violações regulatórias severas ao abrigo do RGPD e da HIPAA.

As soluções tradicionais de proteção de dados (cifra em repouso, controlo de acesso baseado em roles, auditorias de acesso) são necessárias mas insuficientes. O problema fundamental reside no momento da computação: para executar uma query, os dados têm de ser descriptados, ficando expostos em memória ao sistema operativo, ao hypervisor e, em ambiente cloud, ao próprio operador da infraestrutura. Um adversário com acesso privilegiado ao sistema pode contornar todos os controlos lógicos e aceder diretamente à memória do processo em execução.

Este relatório explora como as Trusted Execution Environments (TEEs), e em particular o Intel Software Guard Extensions (SGX), podem servir como primitiva de segurança fundamental para resolver este problema. Ao isolar a execução de código sensível em enclaves protegidos ao nível do hardware, cuja confidencialidade e integridade são garantidas mesmo face ao sistema operativo e ao hypervisor, o SGX permite que um operador cloud não confiável aloje computação sobre dados médicos sensíveis sem jamais ter acesso aos mesmos em claro.

O projeto proposto visa concretizar esta visão num sistema funcional: um data lake médico colaborativo onde os participantes do consórcio submetem registos cifrados, um enclave SGX processa queries SQL-style sobre esses registos, e a atestação remota via Intel DCAP garante que todos os participantes possam verificar criptograficamente a autenticidade e integridade do código em execução antes de partilharem qualquer dado.

2. Contexto

Nos últimos anos, o crescimento exponencial no volume, velocidade e variedade da informação gerada tornou evidente que os sistemas de armazenamento tradicionais, frequentemente rígidos e focados num esquema pré-definido, já não são suficientes. É neste enquadramento de modernização analítica e da necessidade de reter informação na sua forma mais pura que

surge a motivação para a adoção de novas abordagens de armazenamento, preparadas para alimentar modelos de inteligência artificial, *machine learning* e análises de *big data*.

2.1 Data Lakes

Um Data Lake atua como um repositório centralizado projetado para armazenar volumes massivos de dados estruturados, semiestruturados e não estruturados, preservando o seu formato nativo (*raw data*). Ao contrário das bases de dados convencionais, os Data Lakes adotam tipicamente uma abordagem *schema-on-read*, o que significa que não exigem a definição de uma estrutura ou esquema rígido no momento da escrita, oferecendo assim uma enorme flexibilidade.

Este paradigma de armazenamento é fundamental para criar uma camada de abstração entre os sistemas que produzem os dados (como sistemas *legacy*, sensores IoT ou aplicações web) e os sistemas consumidores. No âmbito da Ciência de Dados, esta característica é particularmente vantajosa, pois proporciona aos investigadores e analistas um ecossistema altamente colaborativo, onde é possível explorar livremente os dados em bruto para testar novas hipóteses, sem interferir com as operações diárias das fontes originais.

2.2 Arquitetura Base

A infraestrutura de um Data Lake é desenhada para suportar uma elevada heterogeneidade e escalabilidade. Em termos arquiteturais, estes sistemas são capazes de ingerir e consolidar fluxos de informação provenientes de uma vasta gama de origens, que vão desde sistemas transacionais antigos até *data brokers* externos e APIs de redes sociais.

Para suportar este volume e garantir a eficiência no armazenamento e processamento distribuído, a arquitetura apoia-se frequentemente em frameworks de Big Data como o Apache Hadoop (através do HDFS) ou sistemas de armazenamento em *cloud* baseados em objetos, complementados por motores de computação de alto desempenho, como o Apache Spark. Estas ferramentas formam a espinha dorsal que permite não só reter os conjuntos de dados, mas também orquestrar os processos de exploração e transformação.

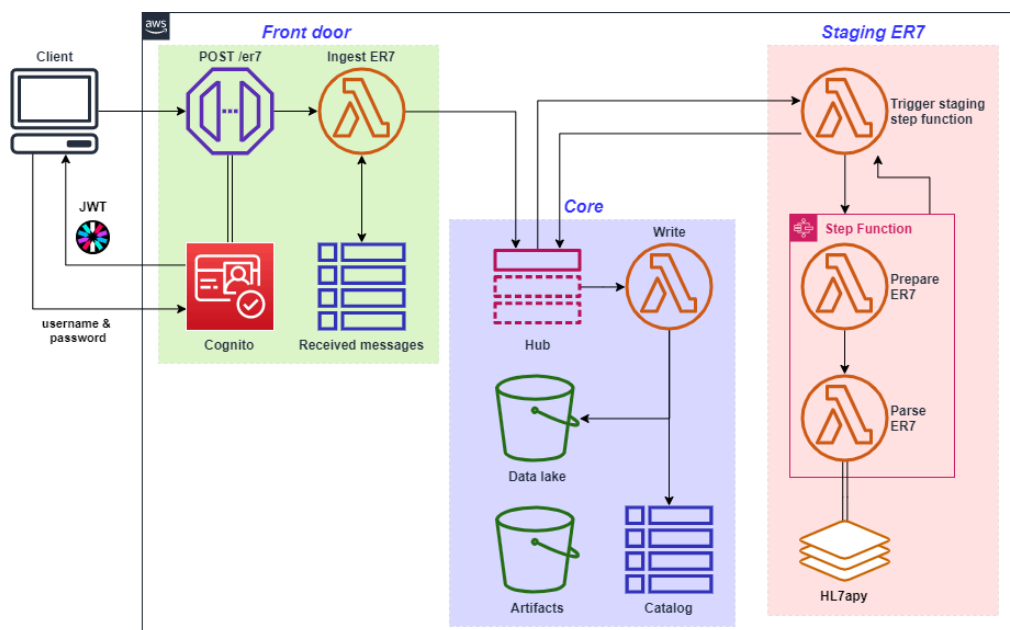


Figura 1 - exemplo de um data lake

3. Fundamentos Técnicos

3.1 Intel Software Guard Extensions (SGX)

O Intel SGX é um conjunto de extensões ao conjunto de instruções x86, introduzido comercialmente na microarquitetura Skylake (2015), que permite a criação de regiões de memória isoladas designadas enclaves. O modelo de segurança do SGX baseia-se numa premissa radical: a confiança é depositada exclusivamente no processador Intel, sendo todos os outros componentes do sistema, sistema operativo, hypervisor, BIOS, outros processos, hardware de memória externo, considerados potencialmente maliciosos.

A proteção do enclave é implementada ao nível do hardware através do Enclave Page Cache (EPC), uma região de memória física cifrada e autenticada pelo processador usando AES-128 em modo CTR para confidencialidade e CBC-MAC para integridade. As chaves de cifra são geradas pelo processador no arranque e nunca saem do chip. Qualquer tentativa de ler a

memória EPC fora do contexto do enclave, por parte do sistema operativo, do hypervisor ou por acesso direto ao barramento de memória, resulta em dados cifrados sem qualquer utilidade para o atacante.

O ciclo de vida de um enclave começa com a sua criação (ECREATE), seguida do carregamento de páginas de código e dados (EADD), da extensão do digest de medição (EEXTEND), e da inicialização que culmina na selagem do enclave com o seu hash de medição (EINIT). Este hash, a measurement ou MRENCLAVE, constitui a identidade criptográfica do enclave e é o valor central verificado durante a atestação remota.

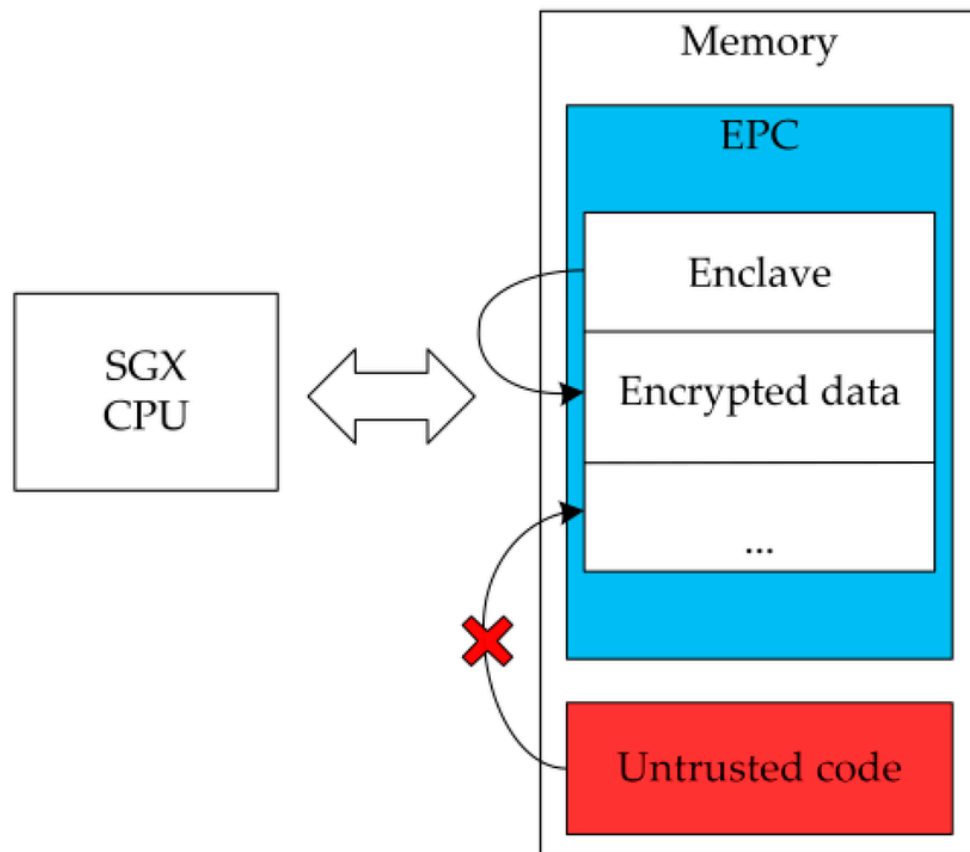


Figure 2 - Modelo de isolamento e fronteira de confiança

3.2 Atestação Remota e DCAP

A atestação remota é o mecanismo que permite a uma entidade externa (um participante do consórcio) verificar que está a comunicar com um enclave legítimo, executando código não modificado, num processador Intel genuíno. Sem atestação, um adversário poderia executar código arbitrário num processo normal e fingir ser o enclave para obter as chaves de cifra dos participantes.

O modelo clássico de atestação SGX (EPID, Enhanced Privacy ID) dependia de servidores centralizados da Intel para a verificação de assinaturas. O DCAP (Data Center Attestation Primitives), introduzido em 2019 para processadores Ice Lake e posteriores com suporte a ECDSA attestation, permite que a verificação seja realizada localmente no datacenter, sem dependência de conectividade com servidores Intel em tempo real.

O fluxo DCAP envolve: (1) o enclave gera um Report SGX que inclui o MRENCLAVE, atributos do enclave, e dados customizados (e.g., uma chave pública efémera para estabelecimento de canal seguro); (2) o Quoting Enclave (QE) assina o Report com a chave de atestação ECDSA derivada da PCK (Provisioning Certification Key) do processador; (3) o verificador usa o serviço PCCS (Provisioning Certificate Caching Service) local para obter os certificados PCK, TCB Info e QE Identity necessários para verificar a cadeia de confiança até à CA raiz da Intel; (4) após verificação bem-sucedida, o verificador sabe que o enclave em questão executa código com o MRENCLAVE declarado, num processador Intel com TCB de nível conhecido.

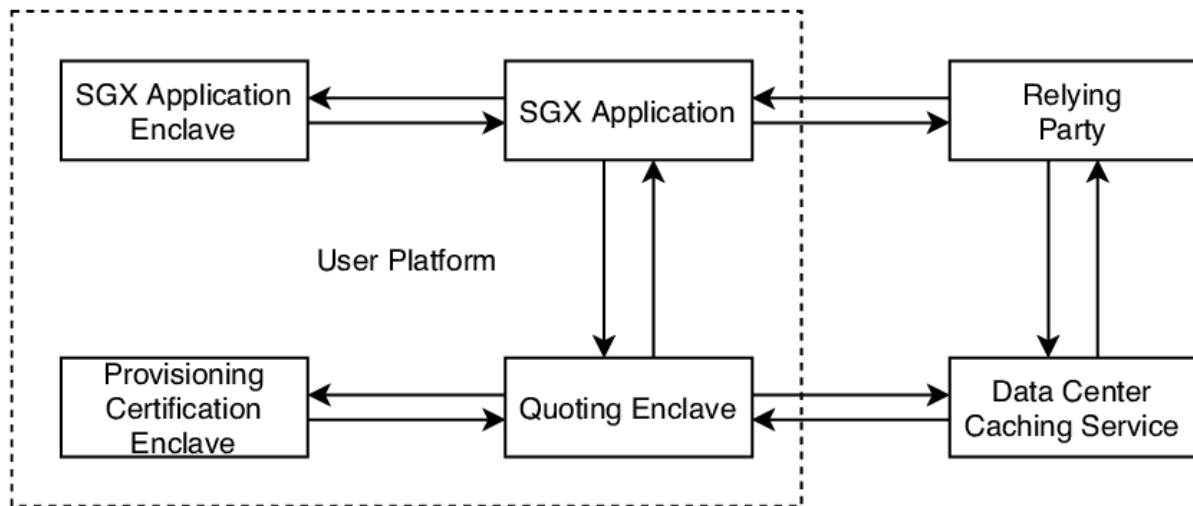


Figura 3 - Fluxo de mensagens

3.3 Criptografia Aplicada ao Enclave

O design criptográfico do sistema baseia-se em primitivas bem estabelecidas. Para o estabelecimento de canal seguro entre participantes e o enclave, usa-se ECDH (Elliptic-Curve Diffie-Hellman) sobre Curve25519, cuja chave pública do enclave é certificada durante a atestação remota. Os registos individuais são cifrados pelos participantes com AES-256-GCM, que oferece tanto confidencialidade como autenticidade, garantindo que quaisquer modificações aos dados cifrados sejam detetadas antes da descriptação no enclave. Os resultados das queries são autenticados com HMAC-SHA256 usando uma chave derivada do segredo partilhado entre o enclave e o participante que solicitou a query.

4. Estado da Arte

4.1 Sistemas de Analytics Confidencial com SGX

A investigação sobre analytics confidencial com SGX tem produzido resultados significativos ao longo da última década. O trabalho seminal nesta área é o Opaque, apresentado por Zheng et al. no NSDI 2017 [3]. O Opaque integra o Apache Spark com SGX para suportar queries SQL sobre dados cifrados distribuídos. A sua contribuição principal é dupla: por um lado, garante confidencialidade e integridade dos dados durante a computação; por outro, introduz um modo oblivious que oculta os padrões de acesso a memória, mitigando ataques de canal lateral baseados em page faults.

O EnclaveDB, apresentado por Priebe et al. no IEEE S&P 2018 [4], aborda o problema de uma perspectiva diferente: em vez de adaptar um motor de analytics distribuído, os autores constroem uma base de dados relacional completa (baseada em Hekaton, o motor in-memory do SQL Server) que executa inteiramente dentro de um enclave SGX. O EnclaveDB garante confidencialidade, integridade e freshness de todas as operações, incluindo proteção contra rollback attacks que tentariam reverter o estado da base de dados para uma versão anterior.

O VC3, apresentado por Schuster et al. no IEEE S&P 2015 [5], foca-se em jobs MapReduce confidenciais na cloud, permitindo que os utilizadores verifiquem que o seu código foi executado corretamente sobre os dados corretos, sem expor nenhum dos dois ao operador cloud. Embora anterior ao Opaque e focado num paradigma de computação diferente, o VC3 estabeleceu muitos dos princípios arquiteturais adotados por trabalhos posteriores.

Mais recentemente, o DuckDB-SGX2 [6], publicado em 2024, investiga o desempenho de um motor OLAP moderno (DuckDB) dentro de enclaves SGX2 usando o Gramine LibOS, executando o benchmark TPC-H padrão. Os resultados são relevantes para o nosso projeto: o overhead médio face à execução nativa é de 2-4x para queries que cabem em EPC, escalando significativamente quando ocorre paginação. Este trabalho valida a abordagem Gramine + motor SQL existente para o nosso contexto.

4.2 Computação Confidencial no Domínio Médico

No domínio específico da saúde, a investigação tem explorado múltiplas abordagens complementares. O SMPC4Health e projetos similares usam computação multi-partes segura para queries federadas sobre dados hospitalares distribuídos, sem centralização dos dados. Embora ofereçam garantias formais de privacidade, estas abordagens têm overhead computacional proibitivo para queries complexas e escalabilidade limitada.

O projeto EHDEN (European Health Data Evidence Network), financiado pela IMI (Innovative Medicines Initiative), demonstrou a viabilidade de análises à escala europeia sobre dados clínicos standardizados no formato OMOP CDM. A infraestrutura EHDEN utiliza uma arquitetura federada, na qual os dados permanecem nos hospitais e apenas os resultados agregados são partilhados, tornando-se assim uma abordagem mais conservadora, mas menos flexível do que a centralização num data lake com TEE

Trabalhos recentes têm explorado a combinação de TEEs com privacidade diferencial para análises médicas confidenciais. A privacidade diferencial, formalizada por Dwork et al. [7], adiciona ruído calibrado aos resultados das queries para limitar a informação que um adversário pode inferir sobre registos individuais, mesmo a partir de múltiplas queries. A integração com TEEs garante que o mecanismo de adição de ruído não pode ser contornado por um operador malicioso.

4.3 Trusted Execution Environments: Comparativo

Além do Intel SGX, outras plataformas TEE relevantes incluem o AMD SEV (Secure Encrypted Virtualization), o ARM TrustZone, e o RISC-V Keystone. O AMD SEV opera ao nível da máquina virtual, cifrando a memória de toda a VM, o que facilita a migração de aplicações existentes mas oferece uma TCB mais extensa e garantias de integridade mais fracas que o SGX (o SEV original não incluía proteção de integridade, apenas confidencialidade; as versões SEV-ES e SEV-SNP adicionaram proteções progressivas). O ARM TrustZone é amplamente

usado em dispositivos móveis mas foi originalmente concebido para casos de uso de DRM e autenticação biométrica, sendo menos adequado para analytics intensiva. O RISC-V Keystone é uma plataforma de investigação que permite customizar o design do TEE, útil para explorar novos modelos de segurança mas sem o ecossistema maduro do SGX.

Para o contexto deste projeto o Intel SGX apresenta o melhor equilíbrio entre maturidade do ecossistema, expressividade das garantias de segurança, e desempenho.

4.4 Ataques Conhecidos ao SGX e Mitigações

O SGX não é imune a ataques. Foram identificadas várias classes de vulnerabilidades que o design do nosso sistema deve considerar. Os ataques de canal lateral baseados em cache (Flush+Reload, Prime+Probe) permitem inferir padrões de acesso à memória do enclave a partir de um processo localizado, potencialmente revelando informação sobre os dados processados. A mitigação principal é o uso de algoritmos oblivious (tempo constante) para operações sensíveis.

Os ataques de page fault (ou controlled-channel attacks) exploram o facto de o sistema operativo controlar as tabelas de páginas: um sistema operativo malicioso pode inferir os padrões de acesso do enclave a granularidade de página (4KB) através da manipulação de page faults. O SGX2 mitiga parcialmente este problema através de EDMM (Enclave Dynamic Memory Management). O Spectre e variantes exploram a execução especulativa e afetam enclaves SGX, embora os patches de microcode e as mitigações de compilador (LFENCE) reduzam significativamente a superfície de ataque.

É importante notar que estes ataques requerem, geralmente, co-localização ou acesso físico ao sistema, e que as versões mais recentes do SGX (Ice Lake em diante) incluem mitigações de hardware para as variantes mais severas. Para o nosso modelo de ameaça, onde o adversário principal é o operador cloud sem acesso físico ao processador, estas mitigações são suficientes.

5. Arquitetura do Sistema

5.1 Visão Geral

A arquitetura do sistema divide-se em quatro componentes principais: (1) os clientes participantes do consórcio, que submetem registos cifrados e emitem queries; (2) o servidor de processamento seguro, que hospeda o enclave SGX; (3) o serviço de atestação e gestão de chaves, que medeia o estabelecimento de confiança entre clientes e enclave; e (4) o serviço de armazenamento persistente, que mantém os registos cifrados em disco.

O princípio central de design é que dados em claro nunca saem do enclave. Os registos são cifrados pelos clientes antes da transmissão, descriptados apenas dentro do enclave, e os resultados das queries são autenticados antes de saírem do enclave. O operador cloud que hospeda o servidor nunca tem acesso a dados em claro, chaves de cifra, ou resultados não autenticados.

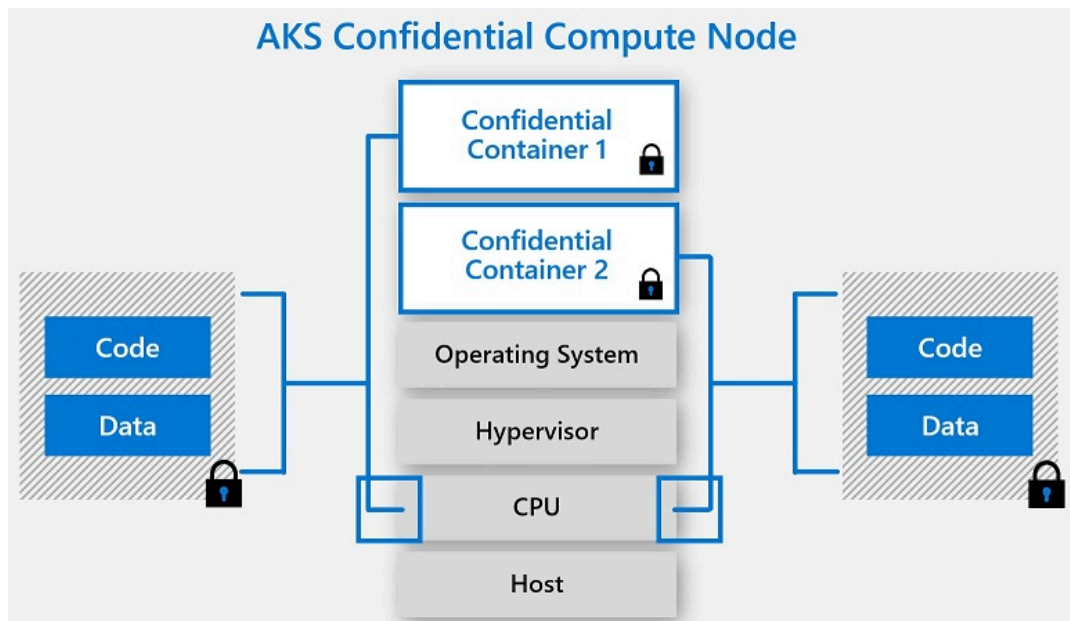


Figura 4 - Arquitetura global do sistema

5.2 Fluxo de Ingestão de Dados

Antes de submeter qualquer dado, cada participante deve completar o protocolo de atestação remota. O cliente solicita um Report ao enclave, que inclui a chave pública ECDH efêmera do enclave e o MRENCLAVE (hash do código do enclave). O cliente verifica o Report usando o DCAP Quote Verification Library (QVL), confirmando: (a) a assinatura ECDSA é válida e encadeia até à CA raiz Intel; (b) o MRENCLAVE corresponde ao hash do código auditado e publicado do enclave; (c) o nível de TCB do processador é adequado (sem vulnerabilidades críticas não *patched*).

Após atestação bem-sucedida, o cliente usa a chave pública do enclave para executar ECDH e derivar uma chave de sessão simétrica (AES-256) via HKDF. Esta chave de sessão é usada para cifrar os registos a submeter. Cada registo é serializado em formato estruturado (e.g., Protocol Buffers), cifrado com AES-256-GCM (incluindo um nonce aleatório e um AD, Additional Data, com metadados do registo), e transmitido ao servidor. O enclave descripta o registo, verifica a autenticidade via GCM tag, e armazena-o em memória protegida ou em disco em formato cifrado com uma chave mestre do enclave (sealed key).

5.3 Processamento de Queries

As queries são submetidas pelos participantes através de um canal seguro TLS, com autenticação mútua. Dentro do enclave, o motor SQL mínimo suporta as seguintes operações: COUNT (contagem de registos por critério), SUM e AVG (soma e média de campos numéricos), GROUP BY (agregação por categoria, e.g., diagnóstico ou faixa etária), e correlação de Pearson entre dois campos numéricos (relevante para análises de eficácia de medicamentos). A implementação usa um motor SQL embedded com as tabelas carregadas em memória a partir dos registos descriptados.

Antes de devolver os resultados, o enclave aplica um conjunto de verificações de segurança. O threshold mínimo de registos garante que não são devolvidos resultados de queries sobre

menos de K registos (e.g., $K=5$), prevenindo a inferência trivial de registos individuais. A verificação de autorização confirma que o participante que emitiu a query tem permissão para aceder às categorias de dados envolvidas. O resultado é assinado com HMAC-SHA256 usando a chave de sessão do participante, garantindo autenticidade e *non-repudiation*.

5.4 Modelo de Confiança e Garantias de Segurança

O sistema oferece as seguintes garantias formais, derivadas das propriedades do SGX e do design arquitetural: confidencialidade (dados individuais nunca são revelados a nenhuma entidade fora do enclave, incluindo o operador cloud); integridade (os resultados das queries refletem fielmente o estado atual dos dados, sem possibilidade de adulteração não detetada); autenticidade (os participantes podem verificar criptograficamente que o enclave executa o código correto, não modificado); e freshness parcial (proteção contra rollback simples através de contadores monotónicos no enclave, embora proteção completa requiera hardware TPM adicional).

6. Implementação e Desenvolvimento

6.1 Stack Tecnológico

A implementação usa uma *library* OS compatível com SGX como ambiente de execução do enclave, permitindo executar aplicações Linux não modificadas dentro de SGX sem necessidade de reestruturação profunda do código.. O motor SQL que queremos explorar é o DuckDB, um motor OLAP in-process com suporte nativo a queries analíticas complexas e desempenho competitivo em benchmarks padrão (TPC-H, TPC-DS). A camada de comunicação usa TLS 1.3 com autenticação mútua por certificados, gerados e assinados dentro do enclave durante a atestação.

A atestação remota usa a biblioteca libsgx-dcap-ql (Quote Generation Library) e a libsgx-dcap-quoteverify (Quote Verification Library), ambas open-source e mantidas pela Intel. O protocolo criptográfico de estabelecimento de chave usa a biblioteca libsodium, que oferece implementações auditadas e resistentes a timing attacks das primitivas necessárias (X25519 ECDH, HKDF, AES-256-GCM, HMAC-SHA512).

6.2 Enclave Mínimo - Fase 1

A primeira fase de implementação foca no desenvolvimento de um enclave mínimo que demonstra o fluxo fundamental: receção de um registo cifrado, descriptação dentro do enclave, execução de uma agregação simples (média de um campo numérico), e devolução do resultado autenticado. Esta fase usa o Intel SGX SDK diretamente para minimizar a complexidade inicial e garantir o controlo total sobre o código do enclave.

O enclave mínimo expõe duas ECalls essenciais ao código não confiável: *ecall_upload_data*, que recebe um lote de registos cifrados com AES-128-GCM (juntamente com o IV e a tag de autenticação), os descripta dentro do enclave usando *sgx_rijndael128GCM_decrypt* e os armazena em memória protegida; e *ecall_run_query*, que executa agregações simples (AVG, MIN, MAX, COUNT) sobre os campos numéricos dos registos, com filtragem opcional por diagnóstico, devolvendo o valor agregado e o número de registos processados. O código não

confiável (a aplicação host) é responsável pelo carregamento dos ficheiros CSV, pela cifragem dos registos antes do upload e pela interface de queries, operações que não requerem acesso a dados em claro. Nesta fase, a chave de sessão é estabelecida de forma simplificada e a atestação remota é deixada para a Fase 4.

6.3 Suporte Multi-Contribuidor - Fase 2

A segunda fase alarga o enclave para suportar múltiplos participantes com sessões independentes. Cada participante tem um contexto de sessão próprio dentro do enclave, com a sua chave de sessão derivada do ECDH com a chave mestre do enclave. O isolamento entre sessões é crítico: um registo submetido pelo Hospital A não deve ser acessível ao Instituto B, exceto através de queries autorizadas sobre o dataset agregado.

A gestão de chaves no enclave usa um keystore interno, onde as chaves de sessão são indexadas por identificador de participante e têm TTL (Time-To-Live) configurável. Quando uma sessão expira ou é terminada, a chave correspondente é sobrescrita com zeros antes de ser libertada. A derivação de chaves de long-term a partir do sealing key do processador permite que o estado do enclave (registos armazenados em disco cifrado) seja persistente entre reinicializações do sistema, sem exigir um novo processo de atestação completo.

6.4 Motor de Queries - Fase 3

A terceira fase integra o DuckDB no ambiente de execução do enclave e implementa o subconjunto de queries SQL necessárias para as análises médicas alvo. O DuckDB é carregado dentro do enclave através da camada de isolamento escolhida, que é responsável por restringir os recursos do sistema acessíveis (ficheiros, syscalls, rede). As tabelas DuckDB são mantidas em memória, com os dados carregados a partir dos registos descriptados no início de cada sessão de query.

Um aspeto crítico desta fase é a prevenção de timing side-channels nas queries: o tempo de resposta de uma query não deve revelar informação sobre os dados processados. Esta propriedade é parcialmente garantida pelo padding das respostas para tamanho fixo e pela adição de latência aleatória calibrada, embora mitigação completa requeira algoritmos oblivious que constituem trabalho futuro.

6.5 Atestação Remota - Fase 4

A quarta fase integra a pipeline completa de atestação DCAP. O servidor disponibiliza acesso a um serviço de aprovisionamento de certificados PCK e TCB Info (seja através de um PCCS local ou de um serviço externo compatível) mantidos atualizados em relação aos servidores Intel. O cliente implementa o protocolo de atestação: solicita um Quote ao servidor, verifica-o usando uma biblioteca de verificação DCAP compatível, extrai os dados do Report Data do enclave (que podem incluir material criptográfico para estabelecimento de sessão), e só então procede ao estabelecimento de canal seguro.

Em ambiente de desenvolvimento sem hardware SGX compatível com DCAP, recorre-se a modos de simulação ou emulação disponibilizados pelo SDK utilizado. O relatório final documentará explicitamente quais componentes foram testados em hardware real versus emulado, com as implicações de segurança correspondentes.

7. Protótipo Implementado

Esta secção descreve o protótipo funcional desenvolvido e publicado num repositório do github [13], correspondente à Fase 1 da implementação descrita na secção 6.2. O objetivo é validar o fluxo fundamental do sistema (receção de dados cifrados, descriptação dentro do enclave e execução de queries agregadas) utilizando diretamente o Intel SGX SDK em modo de simulação.

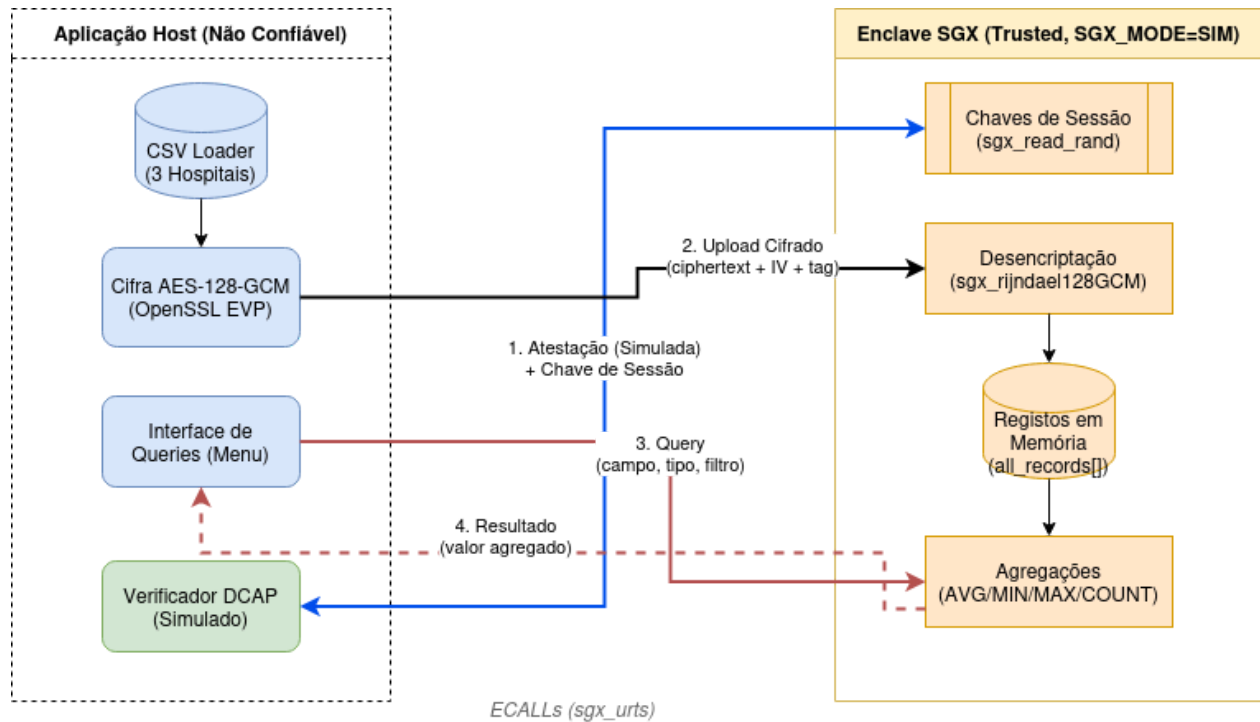


Figura 5 - Diagrama de arquitetura

7.1 Stack Tecnológica

O protótipo foi desenvolvido em C/C++ sobre o Intel SGX SDK para Linux. A decisão de usar o SDK diretamente, em vez de uma library OS como o Gramine, permite controlo total sobre o código do enclave e minimiza a complexidade inicial, facilitando a compreensão da fronteira de confiança.

A compilação e execução decorrem em **modo de simulação** (*SGX_MODE=SIM*), uma vez que a máquina de desenvolvimento não dispõe de hardware SGX. Este modo replica a interface de programação do SDK (ECALLs, OCALLs, bibliotecas criptográficas) mas não oferece as garantias de isolamento de memória do hardware real, pelo que os dados do enclave residem em memória convencional do processo.

No lado não confiável, a cifra dos dados utiliza a biblioteca OpenSSL (AES-128-GCM via interface EVP). No lado confiável, o enclave recorre exclusivamente às primitivas criptográficas do SDK (*sgx_rjndael128GCM_decrypt* para descriptação e *sgx_ecdsa_sign* para assinaturas), evitando dependências externas dentro do ambiente protegido.

O sistema de build usa o *sgx_edger8r* para gerar automaticamente os wrappers de fronteira a partir do ficheiro EDL: *Enclave_t.{c,h}* (stubs confiáveis) e *Enclave_u.{c,h}* (proxies não confiáveis). O enclave é compilado como biblioteca partilhada, assinado com uma chave RSA privada, e carregado pela aplicação host como *enclave.signed.so*.

7.2 Estrutura do Código

O código segue o modelo split-trust do SGX, dividido em dois lados com responsabilidades distintas:

App/	Lado não confiável (aplicação host)
app_main.cpp	Ponto de entrada, menu interativo, init/teardown do enclave
csv_loader.cpp/h	Parsing de ficheiros CSV com dados de pacientes
crypto.cpp/h	Cifra AES-128-GCM via OpenSSL
attestation.cpp/h	Orquestração do fluxo de atestação remota DCAP
upload.cpp/h	Upload de dados cifrados para o enclave
query.cpp/h	Interface interativa de queries agregadas
helpers.cpp/h	Utilidades de formatação e apresentação
hospital_state.h	Estrutura HospitalState, MRENCLAVE esperado
Enclave/	Lado confiável (enclave SGX)
Enclave.cpp	Atestação, troca de chaves, descriptação, queries
Enclave.edl	Definição da interface ECALL/OCALL
Enclave.config.xml	Configuração de memória e threads do enclave
Include/	
types.h	Estruturas partilhadas entre ambos os lados

Figura 6 - Estrutura do protótipo

A modularização do lado não confiável em ficheiros separados permite isolar responsabilidades e facilitar a evolução do código. Cada módulo expõe uma função principal que é invocada pelo menu em *app_main.cpp*.

7.3 Estruturas de Dados e Interface

A estrutura central do sistema é o *PatientRecord*, definida em *Include/types.h* e partilhada entre ambos os lados da fronteira de confiança. Cada registo contém o identificador do paciente, idade, temperatura corporal, glicemia e um código de diagnóstico (saudável, diabetes, hipertensão ou infeção), ocupando 20 bytes.

No lado não confiável, o estado de cada hospital é mantido numa estrutura *HospitalState* que guarda o nome do hospital, o caminho para o ficheiro CSV, os registos carregados em memória, a chave de sessão AES-128 obtida durante a atestação, e flags que indicam se a atestação e o upload foram concluídos. Dentro do enclave, os registos descriptados são armazenados num array global, juntamente com as chaves de sessão indexadas por hospital.

A fronteira de confiança é definida pelo ficheiro *Enclave.edl*, que especifica quatro ECALLs: *ecall_generate_report* para geração do quote DCAP, *ecall_finish_key_exchange* para obtenção da chave de sessão, *ecall_upload_data* para receção e descriptação de dados, e *ecall_run_query* para execução de queries agregadas. O único OCALL definido é *ocall_print_string*, que permite ao enclave solicitar a impressão de mensagens de diagnóstico no stdout. O EDL especifica anotações de tamanho para todos os buffers que cruzam a fronteira, garantindo que o SDK copia apenas a quantidade necessária de dados entre memória confiável e não confiável.

7.4 Fluxo de Atestação Remota

O protótipo implementa um fluxo de atestação DCAP simulado. A aplicação host começa por gerar um nonce aleatório de 16 bytes via *RAND_bytes()* do OpenSSL, garantindo freshness contra ataques de replay.

O nonce é enviado ao enclave através de *ecall_generate_report()*. O enclave constrói um report que inclui o MRENCLAVE (medição simulada do código), o MRSIGNER (medição da chave de assinatura), e o nonce embebido no campo *user_data*. Sobre a concatenação destes campos (96 bytes), o enclave gera uma assinatura ECDSA usando *sgx_ecdsa_sign()*, simulando o papel do Quoting Enclave.

De volta ao lado não confiável, a aplicação verifica o quote em três passos: confirma que o nonce devolvido corresponde ao desafio original (proteção contra replay), compara o MRENCLAVE com o valor esperado hardcoded (confirmação da identidade do código), e verifica o MRSIGNER. A verificação da assinatura do Quoting Enclave é aceite como válida nesta fase, ficando a verificação completa da cadeia de certificados Intel para fases futuras.

Após atestação bem-sucedida, a aplicação invoca *ecall_finish_key_exchange()*. O enclave gera uma chave de sessão aleatória de 16 bytes via *sgx_read_rand()*, armazena-a internamente indexada pelo identificador do hospital, e devolve uma cópia à aplicação host. Esta chave será usada para cifrar os dados antes do upload.

7.5 Cifra e Upload de Dados

O upload de dados segue um fluxo de cifra-transmissão-desencryção. No lado não confiável, os registos de pacientes carregados a partir do ficheiro CSV são cifrados com AES-128-GCM usando a chave de sessão obtida durante a atestação. A função *encrypt_data()* gera um IV aleatório de 12 bytes, cifra o array de registos usando a interface EVP do OpenSSL, e produz o ciphertext acompanhado de um tag de autenticação de 16 bytes.

O ciphertext, IV e tag são passados ao enclave via *ecall_upload_data()*. Dentro do enclave, *sgxrijndael128GCM_decrypt()* descripta os dados usando a chave de sessão armazenada internamente. O tag GCM é verificado automaticamente, garantindo que os dados não foram adulterados em trânsito. Os registos descriptados são armazenados no array global do enclave, e o buffer temporário é limpo com `memset` antes de ser libertado.

7.6 Motor de Queries

O enclave implementa um motor de queries agregadas que opera sobre todos os registos armazenados, independentemente do hospital de origem. A função *ecall_run_query()* aceita três parâmetros: o campo a agregar (idade, temperatura ou glicemia), o tipo de agregação (média, mínimo, máximo ou contagem) e um filtro opcional por diagnóstico.

A execução itera sobre todos os registos, aplica o filtro quando solicitado, e calcula o agregado pedido. Apenas o valor agregado, o total de registos processados e o número de registos correspondentes ao filtro são devolvidos à aplicação host, sem nunca expor registos individuais. Esta propriedade é central ao modelo de privacidade do sistema, uma vez que os dados em claro existem apenas dentro do enclave e o exterior recebe unicamente resultados estatísticos.

7.7 Execução do Protótipo

O protótipo configura três hospitais (Hospital Santa Maria, Hospital São João e Hospital Santo António), cada um associado a um ficheiro CSV com dados de exemplo. O menu principal permite executar cada passo individualmente (carregar CSV, atestar, fazer upload, executar query) ou correr o fluxo completo para os três hospitais de uma só vez. Após o upload dos três hospitais, as queries agregam dados de todas as fontes no enclave, demonstrando a análise segura multi-hospital sem exposição de dados individuais.

7.8 Simplificações Face à Arquitetura Alvo

O protótipo adota várias simplificações relativamente à arquitetura descrita na secção 5, consistentes com os objetivos da Fase 1.

A nível criptográfico, a troca de chaves não usa ECDH. O enclave gera a chave de sessão unilateralmente e devolve-a em claro, em vez do estabelecimento mútuo com derivação via HKDF previsto na arquitetura alvo. A cifra simétrica usa AES-128-GCM em vez de AES-256-GCM, e os resultados das queries não são autenticados com HMAC.

A atestação DCAP é simulada, com o MRENCLAVE hardcoded em vez de ser calculado pelo processador, e a assinatura do Quoting Enclave aceite sem verificação da cadeia de certificados Intel. Em hardware real, esta verificação encadearia até à CA raiz da Intel via o serviço PCCS.

A nível arquitetural, o protótipo funciona como processo único (host e enclave no mesmo processo), sem a separação cliente/servidor com TLS prevista para fases posteriores. Os dados existem apenas em memória protegida do enclave, sem persistência em disco com sealing key. O motor de queries implementa agregações simples (AVG, MIN, MAX, COUNT) de forma manual, em vez de recorrer ao DuckDB via Gramine. Não existe threshold mínimo de registos por resultado (k-anonymity), pelo que todas as queries são respondidas independentemente do número de registos correspondentes.

Estas simplificações são deliberadas: o protótipo foca-se na validação do fluxo end-to-end (atestação → cifra → upload → descriptação → query) e na familiarização com a API do SGX SDK, delegando as funcionalidades mais avançadas para fases posteriores do desenvolvimento.

8. Avaliação de Segurança e Desafios

8.1 Análise de Ameaças por Componente

A análise de segurança do sistema considera cada componente em relação ao modelo de ameaça definido. O canal TLS entre cliente e servidor é protegido contra ataques de rede por cifra e autenticação mútua. O servidor (código não confiável) não tem acesso a dados em claro; mesmo comprometido, apenas pode realizar ataques de negação de serviço (rejeitar submissões ou queries) ou de rollback (responder com dados antigos, detetável via freshness mechanisms). O enclave é protegido pelo hardware SGX contra leitura direta da memória, sendo o vetor de ataque principal os side-channels.

8.2 Limitações do EPC e Paginação

Uma limitação operacional significativa do SGX é o tamanho do EPC. Em processadores pré-Ice Lake, o EPC é limitado a 128 MB (92 MB utilizáveis após overhead do hardware). Quando o enclave excede este limite, o hardware realiza paginação de EPC: páginas são cifradas e escritas para memória convencional, sendo recarregadas quando necessário. Este processo tem overhead severo, latência por page fault de 1-40 microsegundos, e pode revelar padrões de acesso a memória ao sistema operativo (controlled-channel attacks).

Para datasets médicos de dimensão moderada (dezenas de milhar de registos com campos numéricos), o dataset cabe confortavelmente em 92 MB. Para datasets maiores, a arquitetura deve ser alargada para suportar processamento em chunks, onde apenas um subconjunto dos

dados é carregado no enclave de cada vez. Os processadores Ice Lake e posteriores alargam o EPC até 512 GB, eliminando este problema em hardware de datacenter moderno.

8.3 Side-Channels e Mitigações

Os ataques de canal lateral representam a classe de ameaças mais sofisticada ao SGX. O Flush+Reload e o Prime+Probe exploram o estado da cache L3 partilhada entre o enclave e outros processos no mesmo core físico para inferir padrões de acesso do enclave. O Spectre Variant 2 (branch target injection) permite que código não confiável controle a execução especulativa dentro do enclave, podendo vaziar dados através de canais de cache.

As mitigações implementadas no projeto incluem: uso da flag **-mindirect-branch=thunk** do GCC para mitigar Spectre V2 [12]; uso de operações em tempo constante (constant-time) para comparações de segurança (comparação de MACs, verificação de chaves); e padding de respostas para ocultar timing de queries. Mitigações completas contra todos os side-channels conhecidos (oblivious RAM, oblivious data structures) estão fora do scope deste projeto mas são identificadas como trabalho futuro.

8.4 Ataques por Inferência Estatística

Mesmo com garantias de confidencialidade individuais, um adversário que possa submeter queries arbitrárias pode inferir dados individuais através de queries cuidadosamente construídas. Por exemplo, a query 'qual a média de HbA1c de doentes do sexo masculino com idade 45, diagnóstico X, hospital Y' pode, se existir apenas um registo que satisfaça esses critérios, revelar o valor individual desse doente.

A mitigação implementada é o threshold mínimo de K registos por resultado (K=5 por omissão, configurável). Esta abordagem, conhecida como k-anonymity parcial, é simples e eficaz para os casos mais óbvios, mas não oferece garantias formais contra adversários adaptativos que combinam múltiplas queries. A integração de privacidade diferencial (mecanismo de Laplace ou de Gauss para respostas numéricas) é identificada como extensão natural e prioritária do sistema.

9. Conclusão

Este trabalho apresentou o design e a implementação de um protótipo de data lake seguro para análise colaborativa de dados médicos entre múltiplos hospitais, utilizando Intel SGX como primitiva de segurança fundamental. A arquitetura proposta garante que os dados individuais dos pacientes nunca são expostos fora do enclave, permitindo a execução de queries agregadas sobre o dataset combinado sem comprometer a confidencialidade dos participantes.

O protótipo desenvolvido valida o fluxo end-to-end do sistema: atestação remota simulada com verificação de MRENCLAVE e nonce, estabelecimento de chaves de sessão por hospital, cifra e upload de registros com AES-128-GCM, descriptação dentro do enclave com verificação de integridade via tag GCM, e execução de queries agregadas (AVG, MIN, MAX, COUNT) com filtragem por diagnóstico. A separação entre código confiável e não confiável, imposta pelo modelo split-trust do SGX, demonstra que é possível processar dados sensíveis num ambiente onde o operador da infraestrutura não tem acesso aos dados em claro.

As limitações do protótipo atual são deliberadas e consistentes com uma primeira fase de desenvolvimento. A execução em modo de simulação não oferece isolamento real de memória, a troca de chaves é unilateral, a atestação não verifica a cadeia de certificados Intel, e os dados não persistem entre sessões. Estas simplificações delimitam o alcance das conclusões de segurança que podem ser retiradas do protótipo, mas não invalidam a validação do fluxo funcional e da viabilidade da abordagem.

O passo seguinte consiste na migração para hardware SGX real, integrando atestação DCAP completa com verificação da cadeia de certificados via PCCS. A introdução de ECDH para troca de chaves mútua, sealing para persistência cifrada em disco, e um threshold de k-anonymity nos resultados das queries completariam progressivamente as garantias de segurança descritas na arquitetura alvo deste relatório.

10. Referências

- [1] Costan, V. & Devadas, S. (2016). Intel SGX Explained. IACR Cryptology ePrint Archive, Report 2016/086. Disponível em: <https://eprint.iacr.org/2016/086.pdf>
- [2] McKeen, F. et al. (2013). Innovative Instructions and Software Model for Isolated Execution. Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy (HASP), Article 10. <https://dl.acm.org/doi/10.1145/2487726.2488368>
- [3] Zheng, W., Dave, A., Beekman, J. G., Popa, R. A., Gonzalez, J. E., & Stoica, I. (2017). Opaque: An Oblivious and Encrypted Distributed Analytics Platform. Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 283-298. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/zheng>
- [4] Priebe, C., Vaswani, K., & Costa, M. (2018). EnclaveDB: A Secure Database Using SGX. Proceedings of the IEEE Symposium on Security and Privacy (S&P), pp. 264-278. <https://ieeexplore.ieee.org/document/8418608>
- [5] Schuster, F., Costa, M., Fournet, C., Gkantsidis, C., Peinado, M., Mainar-Ruiz, G., & Russinovich, M. (2015). VC3: Trustworthy Data Analytics in the Cloud using SGX. Proceedings of the IEEE Symposium on Security and Privacy (S&P), pp. 38-54. <https://ieeexplore.ieee.org/document/7163017>
- [6] Fuhry, B., Kemper, A., & Neumann, T. (2024). DuckDB-SGX2: The Good, The Bad and The Ugly within Confidential Analytical Query Processing. arXiv preprint arXiv:2405.11988. <https://arxiv.org/abs/2405.11988>
- [7] Dwork, C., McSherry, F., Nissim, K., & Smith, A. (2006). Calibrating Noise to Sensitivity in Private Data Analysis. Proceedings of the 3rd Theory of Cryptography Conference (TCC), Lecture Notes in Computer Science, vol. 3876, pp. 265-284.
- [8] Tsai, C.-C., Porter, D. E., & Vij, M. (2017). Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX. Proceedings of the 2017 USENIX Annual Technical Conference (ATC), pp. 645-658. <https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai>
- [9] Intel Corporation. (2022). Intel SGX Data Center Attestation Primitives (DCAP) — ECDSA Orientation Guide. Intel Developer Zone. https://download.01.org/intel-sgx/latest/dcap-latest/linux/docs/DCAP_ECDSA_Orientation.pdf
- [10] Murray, T. et al. (2020). Formal Foundations for Intel SGX Data Center Attestation Primitives. International Conference on Formal Engineering Methods (ICFEM), Lecture Notes in Computer Science, vol. 12531. https://dl.acm.org/doi/abs/10.1007/978-3-030-63406-3_16
- [11] Feng, X. et al. (2024). Survey of Research on Confidential Computing. IET Communications. <https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/cmu2.12759>
- [12] Kocher, P. et al. (2019). Spectre Attacks: Exploiting Speculative Execution. IEEE S&P. Disponível em: <https://spectreattack.com/spectre.pdf>
- [13] afaturc. (2025). SAHC Project: Secure Aggregation for Healthcare Consortiums. GitHub. <https://github.com/afaturc/sahc-project>